PHYSICS-INFORMED AND ROBUST VARIATIONAL NEURAL NETWORKS FOR DIFFUSION–ADVECTION PROBLEMS WITH A DISCONTINUOUS ADVECTION COEFFICIENT

A Project Report Submitted

for the Course

MA4998P Project I

by

Satyaki Ray

(Roll No. 230123080)

Ankit Raj

(Roll No. 230123007)



to the

DEPARTMENT OF MATHEMATICS

INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

GUWAHATI - 781039, INDIA

November 2026

CERTIFICATE

This is to certify that the work contained in this project report entitled "Physics-Informed and Robust Variational Neural Networks for Diffusion–Advection Problems with a Discontinuous Advection Coefficient" submitted by Satyaki Ray (Roll No.: 230123080) and Ankit Raj (Roll No.: 230123007) to the Department of Mathematics, Indian Institute of Technology Guwahati towards partial requirement of Bachelor of Technology in Mathematics and Computing has been carried out by them under my supervision.

It is also certified that, along with literature survey, computational implementations have been carried out and simulation studies have been carried out by the students under the project.

Turnitin Similarity: __ % and AI content: __ %

Guwahati - 781039 (Satyaki Ray, Ankit Raj)
November 2026 Project Students

Guwahati - 781039 (Dr. Bhupen Deka)
November 2026 Project Supervisor

ABSTRACT

The main aim of the project is to study and implement neural network methods for partial differential equations, namely physics-informed neural networks (PINNs), variational PINNs (VPINNs) and robust variational PINNs (RVPINNs), and to apply them to a one-dimensional diffusion–advection problem with a discontinuous advection coefficient. PINNs minimise the strong residual of the equation, VPINNs the weak residual tested against a finite set of functions, and RVPINNs the discrete dual norm of the weak residual, which is equivalent to the energy-norm error under suitable assumptions.

We first reproduce the pure diffusion benchmark of Rojas et al. [12]: all methods reach relative energy errors below 3×10^{-3}, and the RVPINN loss matches the true error with an effectivity index between 0.975 and 1. For a piecewise constant advection coefficient we show that the interface conditions are natural in the weak formulation and that the bilinear form stays coercive when the coefficient does not increase across the interface. Using an exact solution with a boundary layer and an interface layer, we find that the RVPINN loss remains a reliable error estimator provided the test space resolves these layers; with a coarse test space it becomes too optimistic late in training, and a richer test space restores robustness and reduces the error to 3.5×10^{-3}. We explain why the second derivative and the strong residual of variationally trained networks are largest near the layers, and show that an interface-enriched network reduces the energy errors of PINN and VPINN to about 5×10^{-4}.

Contents

List of Figures **vi**

List of Tables **viii**

1 Introduction **1**

1.1 Physics-informed and variational neural networks 1

1.2 Objectives of the project 2

1.3 Contributions 3

1.4 Organisation of the report 4

2 Model Problem and Weak Formulation **5**

2.1 The diffusion–advection problem 5

2.2 Weak formulation 7

2.3 Norms and well-posedness 8

3 Neural Network Methods: PINN, VPINN and RVPINN **11**

3.1 Neural network approximation 11

3.1.1 An interface-enriched network 12

3.2 Physics-informed neural networks (PINN) 13

3.3 Variational PINNs (VPINN) 13

3.4 Robust variational PINNs (RVPINN) 14

3.4.1 Finite element test functions 15
3.4.2 Spectral test functions 15
3.5 Error estimates for RVPINNs 15
3.6 Why the strong residual is not controlled by weak losses . . . 16
3.7 Implementation details 17

4 Results I: Pure Diffusion Benchmark ($\beta = 0$) 19
4.1 Accuracy 19
4.2 The RVPINN loss as an error estimator 21
4.3 Second derivative and strong residual 23

5 Results II: Discontinuous Advection Coefficient 25
5.1 Test problem and exact solution 25
5.2 Results with the standard network 27
5.3 Reliability of the RVPINN error estimate 30
5.4 Results with the interface-enriched network 33
5.5 Discussion 35

6 Conclusions and Future Work 37
6.1 Conclusions 37
6.2 Future work 38

A Code and Reproducibility 40

Bibliography 42

List of Figures

4.1 Pure diffusion benchmark: solutions, pointwise error, second-derivative error and relative energy-norm error during training for the four methods. 20

4.2 RVPINN with 50 spectral test functions for the pure diffusion benchmark: (a) solution, (b) pointwise error, (c) second-derivative error and strong residual (identical here since $\varepsilon = 1$, $\beta = 0$), (d) true relative energy error and loss-based estimate during training. 22

4.3 Pure diffusion benchmark. (a) True relative energy error against $\sqrt{\mathscr{L}}/\|u\|_U$ at every 10th iteration; the dashed line is "error = estimate". (b) Effectivity index $\sqrt{\mathscr{L}}/\|u - u_\theta\|_U$ of the RVPINN losses. 23

5.1 Exact solution of the interface problem with $\varepsilon = 0.1$, $\beta_- = 2$, $\beta_+ = 1$, $f = 1$: u, u' and u''. The vertical line marks the interface $x = 0$. 27

5.2 Discontinuous advection, standard network, 20000 iterations: solutions, pointwise error, second-derivative error and convergence of the relative energy error. 28

5.3 RVPINN with 50 spectral test functions, standard network, 20000 iterations: (a) solution, (b) pointwise error, (c) second-derivative error and strong residual, (d) true error and loss-based estimate. 29
5.4 Discontinuous advection, standard network, $M = 50$ sines and $M = 100$ hats: (a) true relative energy error against the loss-based estimate, (b) effectivity index of the RVPINN losses. . . 31
5.5 As Figure 5.4, but with $M = 200$ sines and $M = 400$ hats. The estimate remains reliable during the whole training. . . . 32
5.6 Discontinuous advection, interface-enriched network, 20000 iterations: solutions, pointwise error, second-derivative error and convergence. 34

List of Tables

4.1 Pure diffusion benchmark, 6000 iterations: relative errors in the energy norm, L^2 norm and second derivative, and RMS of the strong residual (overall and in the boundary region $|x| > 0.9$ and the bulk $0.1 \leq |x| \leq 0.9$). 20

5.1 Discontinuous advection, standard network: relative errors and RMS strong residual near the boundary ($|x| > 0.9$), near the interface ($|x| < 0.1$) and in the bulk, after 6000 and 20000 iterations. 27

5.2 RVPINN with the standard network after 20000 iterations: effect of the size M of the test space on the energy error and on the effectivity index $\sqrt{\mathscr{L}}/\left\|u - u_\theta\right\|_U$. 32

5.3 Discontinuous advection, interface-enriched network 33

Chapter 1

Introduction

Partial differential equations (PDEs) are the standard language for modelling transport, diffusion and many other physical processes. Classical numerical methods such as finite differences and finite elements solve them on a mesh and come with a mature convergence theory [2]. In recent years, neural networks have been proposed as an alternative, mesh-free way to represent the solution of a PDE. The idea goes back to Lagaris et al. [9] and was popularised under the name *Physics-Informed Neural Networks* (PINNs) by Raissi, Perdikaris and Karniadakis [11].

1.1 Physics-informed and variational neural networks

In a PINN, a neural network u_θ with trainable parameters θ is used as a trial function, and θ is chosen to minimise the mean squared *strong-form residual* of the PDE at a set of collocation points, together with penalty terms for boundary conditions. Derivatives of the network are obtained exactly by automatic differentiation. PINNs are simple to implement and have

been applied to a wide range of problems [3], but they also have well-known weaknesses [8]:

- the strong residual requires the solution to be twice differentiable (for second-order problems), which is not the case for problems with discontinuous coefficients or interfaces;
- the value of the loss is not, in general, a reliable measure of the actual error of the approximation.

Variational PINNs (VPINNs), introduced by Kharazmi, Zhang and Karniadakis [5, 6], replace the strong residual by the *weak residual*: the PDE is multiplied by a finite set of test functions and integrated by parts, so that only first derivatives of the network are required. This makes VPINNs naturally suited to problems whose solutions have low regularity. However, the loss of a classical VPINN depends strongly on the choice (and even the scaling) of the test functions [1, 12].

Rojas et al. [12] proposed *Robust* VPINNs (RVPINNs), in which the loss is the discrete dual norm of the weak residual, computed with the Gram matrix of the test space. With this choice the loss becomes independent of the basis chosen for the test space and, under suitable assumptions, its square root is equivalent to the true error in the energy norm. In other words, the loss becomes a reliable and efficient *a posteriori* error estimator.

1.2 Objectives of the project

The objectives of this project were:

1. to study the formulation of PINNs, VPINNs and RVPINNs following [11, 5, 12];

2. to implement these methods for the one-dimensional diffusion–advection model problem (problem (41) of [12]), first in the pure diffusion case $\beta = 0$;

3. to extend the implementation to the case of a *piecewise constant, discontinuous* advection coefficient β with an interface at $x = 0$, as suggested by the project supervisor;

4. to compare the accuracy of the methods, to test whether the RVPINN loss remains a reliable error estimator when β is discontinuous, and to explain the behaviour of the second derivative and of the strong-form residual of the trained networks.

1.3 Contributions

The main contributions of this work are:

- a single, self-contained PyTorch [10] implementation of PINN, VPINN and RVPINN (with finite element and spectral test spaces) for the interface problem, with an analytical exact solution used for verification;

- a short analysis of the well-posedness of the weak formulation when β is discontinuous (Lemma 2.3.1), showing that the coercivity used in [12] holds if $\beta_- \geq \beta_+$ and can fail otherwise;

- a numerical study showing that the RVPINN loss is still an accurate estimator of the energy-norm error in this setting, and that the large second-derivative errors observed in our first experiments are caused by the interface and by the boundary layer, which a smooth (`tanh`) network cannot resolve in H^2;

- the observation that, with a coarse test space, the RVPINN estimate loses its reliability late in training (as predicted by the oscillation term of the theory), and that a richer test space restores it and improves the accuracy;

- an interface-enriched network architecture which allows a jump in the second derivative at the interface and reduces the energy error of PINN and VPINN by factors of about 30 and 12, respectively.

1.4 Organisation of the report

Chapter 2 introduces the model problem, its weak formulation and its well-posedness. Chapter 3 describes PINNs, VPINNs and RVPINNs and the details of our implementation. Chapter 4 reports results for the pure diffusion benchmark ($\beta = 0$), and Chapter 5 reports results for the discontinuous advection coefficient. Chapter 6 summarises the findings and outlines future work. Appendix A describes how to run the code.

Chapter 2

Model Problem and Weak Formulation

In this chapter we state the model problem, derive its weak (variational) formulation and discuss its well-posedness. Throughout, $(\cdot,\cdot)$ denotes the $L^2(-1,1)$ inner product and $H_0^1(-1,1)$ the usual Sobolev space of functions with square-integrable weak derivative that vanish at $x = \pm 1$ [4].

2.1 The diffusion–advection problem

Following [12, Section 5.1], we consider the diffusion–advection problem

$$-(\varepsilon u')' + \beta u' = f \quad \text{in } \Omega = (-1,1), \qquad u(-1) = u(1) = 0, \tag{2.1}$$

where $\varepsilon > 0$ is the diffusion coefficient, β the advection coefficient and f a given source term. Two cases are studied in this project.

1. **Pure diffusion**, $\beta = 0$, $\varepsilon = 1$ (the benchmark of [12, Section 5.4]). Then (2.1) reduces to $-u'' = f$.

2. **Discontinuous advection.** β is piecewise constant with an interface at $x = 0$:

$$\beta(x) = \begin{cases} \beta_-, & -1 < x < 0, \\ \beta_+, & 0 < x < 1, \end{cases} \qquad \beta_- \neq \beta_+. \tag{2.2}$$

In the second case the equation is understood on each subinterval,

$$-\varepsilon u''(x) + \beta(x)u'(x) = f(x), \qquad x \in (-1, 0) \cup (0, 1), \tag{2.3}$$

together with the boundary conditions $u(\pm 1) = 0$ and the two *interface conditions*

$$\begin{aligned} u(0^-) &= u(0^+) \quad \text{(continuity of the solution)}, \\ \varepsilon u'(0^-) &= \varepsilon u'(0^+) \quad \text{(continuity of the diffusive flux)}. \end{aligned} \tag{2.4}$$

Since ε is constant, the second condition simply says that u' is continuous at $x = 0$.

Remark 2.1.1. Even though u and u' are continuous, the second derivative of the solution is in general *discontinuous* at the interface. Indeed, if f is continuous at $x = 0$, subtracting the two one-sided limits of (2.3) and using $u'(0^-) = u'(0^+)$ gives

$$\varepsilon \left[u''\right](0) := \varepsilon\big(u''(0^+) - u''(0^-)\big) = (\beta_+ - \beta_-)\, u'(0). \tag{2.5}$$

Hence $u \notin C^2(-1, 1)$ unless $u'(0) = 0$, and u'' is only piecewise smooth. This has important consequences for methods based on the strong form of the PDE (see Chapters 3 and 5).

2.2 Weak formulation

Let $V := H_0^1(-1,1)$. Multiplying (2.3) by a test function $v \in V$, integrating over each subinterval and integrating the diffusion term by parts, we obtain

$$\int_{-1}^{0} -\varepsilon u'' v \, dx = \int_{-1}^{0} \varepsilon u' v' \, dx - \varepsilon u'(0^-) v(0),$$
$$\int_{0}^{1} -\varepsilon u'' v \, dx = \int_{0}^{1} \varepsilon u' v' \, dx + \varepsilon u'(0^+) v(0),$$

where we used $v(\pm 1) = 0$. Adding the two identities, the interface terms combine into $\varepsilon\,[u'](0)\,v(0)$, which vanishes by the flux condition (2.4). We arrive at the following problem.

Definition 2.2.1 (Weak formulation)**.** Find $u \in U := V = H_0^1(-1,1)$ such that

$$r(u,v) := \ell(v) - a(u,v) = 0 \qquad \forall v \in V, \tag{2.6}$$

where

$$a(u,v) := \varepsilon\,(u',v') + (\beta u', v), \qquad \ell(v) := (f,v). \tag{2.7}$$

For a given $w \in U$, the linear functional $r(w,\cdot) \in V'$ is called the *weak residual*.

Proposition 2.2.2. *Let u be piecewise C^2 on $[-1,0]$ and $[0,1]$. Then u satisfies* (2.3)*, $u(\pm 1) = 0$ and the interface conditions* (2.4) *if and only if $u \in H_0^1(-1,1)$ and u satisfies* (2.6)*.*

Proof. The "only if" part is the computation above: continuity of u ensures $u \in H_0^1(-1,1)$, and the flux condition removes the interface term. Conversely, let $u \in H_0^1(-1,1)$ be piecewise C^2 and satisfy (2.6). Since $u \in H^1$ in one dimension, u is continuous, so the first interface condition holds. Taking v supported in $(-1,0)$ or in $(0,1)$ and integrating by parts back shows that

(2.3) holds in each subinterval. Using this in (2.6) for a general $v \in V$ leaves only $\varepsilon\,[u'](0)\,v(0) = 0$; choosing v with $v(0) \neq 0$ gives $[u'](0) = 0$, i.e. the flux condition. □

Thus the interface conditions are *natural* for the weak formulation: they do not have to be imposed explicitly. This is one of the main reasons for using variational methods for interface problems.

2.3 Norms and well-posedness

As in [12], we equip $U = V$ with the energy norm

$$\|v\|_U^2 = \|v\|_V^2 := \varepsilon\,(v', v') = \varepsilon \int_{-1}^{1} |v'|^2\,dx. \tag{2.8}$$

By the Poincaré inequality on $(-1, 1)$, $\|v\|_{L^2} \le \frac{2}{\pi}\,\|v'\|_{L^2}$ for $v \in V$, so (2.8) is a norm on V equivalent to the full H^1 norm.

When β is a *constant*, $(\beta v', v) = \frac{\beta}{2}\big[v^2\big]_{-1}^{1} = 0$ for every $v \in V$, so $a(v, v) = \|v\|_V^2$ and the problem is coercive with constant one. This is the property used in [12, eq. (46)–(47)], which relies on $\operatorname{div}\beta = 0$. For a discontinuous β this is no longer true, and we need the following lemma.

Lemma 2.3.1 (Advection term for a piecewise constant β)**.** *Let β be given by* (2.2)*. For every $v \in V$,*

$$(\beta v', v) = \frac{\beta_- - \beta_+}{2}\,v(0)^2, \qquad \textit{hence} \qquad a(v, v) = \|v\|_V^2 + \frac{\beta_- - \beta_+}{2}\,v(0)^2. \tag{2.9}$$

Consequently:

1. *if $\beta_- \ge \beta_+$, then $a(v, v) \ge \|v\|_V^2$ for all $v \in V$;*

2. *if* $\beta_- < \beta_+$, *then* $a(v,v) \geq \left(1 - \dfrac{\beta_+ - \beta_-}{4\varepsilon}\right) \|v\|_V^2$, *which is a coercivity estimate provided* $\beta_+ - \beta_- < 4\varepsilon$.

Proof. Since $vv' = \frac{1}{2}(v^2)'$ and $v(\pm 1) = 0$,

$$(\beta v', v) = \beta_- \int_{-1}^{0} \tfrac{1}{2}(v^2)'\,dx + \beta_+ \int_{0}^{1} \tfrac{1}{2}(v^2)'\,dx = \frac{\beta_-}{2} v(0)^2 - \frac{\beta_+}{2} v(0)^2,$$

which proves (2.9), and part 1 follows immediately. For part 2, write $v(0) = \int_{-1}^{0} v'\,dx = -\int_{0}^{1} v'\,dx$. By the Cauchy–Schwarz inequality, $v(0)^2 \leq \int_{-1}^{0} |v'|^2 dx$ and $v(0)^2 \leq \int_{0}^{1} |v'|^2 dx$. Averaging the two bounds gives $v(0)^2 \leq \frac{1}{2}\int_{-1}^{1} |v'|^2 dx = \frac{1}{2\varepsilon}\|v\|_V^2$. Inserting this into (2.9) with $\beta_- - \beta_+ < 0$ gives the claim. □

Lemma 2.3.2 (Boundedness)**.** *For all* $w, v \in V$, $|a(w,v)| \leq \mu \|w\|_U \|v\|_V$ *with* $\mu = 1 + \dfrac{2\|\beta\|_\infty}{\pi\varepsilon}$.

Proof. By Cauchy–Schwarz, $|\varepsilon(w', v')| \leq \|w\|_U \|v\|_V$ and, using the Poincaré inequality,

$$\begin{aligned} |(\beta w', v)| &\leq \|\beta\|_\infty \|w'\|_{L^2} \|v\|_{L^2} \leq \frac{2\|\beta\|_\infty}{\pi} \|w'\|_{L^2} \|v'\|_{L^2} \\ &= \frac{2\|\beta\|_\infty}{\pi\varepsilon} \|w\|_U \|v\|_V . \end{aligned}$$

Adding the two bounds gives the result. □

Theorem 2.3.3 (Well-posedness)**.** *Let* $f \in L^2(-1,1)$ *and* $\beta_- \geq \beta_+$ *(or, more generally,* $\beta_+ - \beta_- < 4\varepsilon$*). Then problem* (2.6) *has a unique solution* $u \in V$, *and* $\|u\|_U \leq \alpha^{-1} \|\ell\|_{V'}$, *where* $\alpha = 1$ *in the first case and* $\alpha = 1 - (\beta_+ - \beta_-)/(4\varepsilon)$ *in the second.*

Proof. By Lemmas 2.3.1 and 2.3.2, $a(\cdot,\cdot)$ is a bounded and coercive bilinear form on the Hilbert space V, and ℓ is a bounded linear functional. The result follows from the Lax–Milgram theorem [4]. □

Remark 2.3.4. When $\beta_+ - \beta_- \geq 4\varepsilon$ the bilinear form may fail to be coercive. Problem (2.3)–(2.4) is still uniquely solvable (it is a linear boundary value problem for an ODE whose solution can be written down explicitly, see Section 5.1), and the inf–sup condition [12, eq. (3)] still holds, but with a constant $\alpha < 1$ that is not given by Lemma 2.3.1. For this reason, all experiments in this report use $\beta_- > \beta_+$, for which $\alpha = 1$ and the theory of [12] applies directly. With $\alpha = 1$ and μ from Lemma 2.3.2, the error estimates of Section 3.5 hold with explicit constants.

Chapter 3

Neural Network Methods: PINN, VPINN and RVPINN

All three methods approximate the solution u by a neural network u_θ and choose the parameters θ by minimising a loss functional $\mathscr{L}(u_\theta)$. They differ only in the definition of the loss. In this chapter we describe the network, the three losses, the theoretical properties of the RVPINN loss, and our implementation.

3.1 Neural network approximation

We use a fully connected feed-forward network $N_\theta : \mathbb{R} \to \mathbb{R}$ with L hidden layers. With $z^{(0)} = x$,

$$z^{(l)} = \sigma\big(W^{(l)} z^{(l-1)} + b^{(l)}\big), \quad l = 1, \dots, L, \qquad N_\theta(x) = W^{(L+1)} z^{(L)} + b^{(L+1)}, \tag{3.1}$$

where $\sigma = \tanh$ and θ collects all weights $W^{(l)}$ and biases $b^{(l)}$. As in [12, Section 5.3] we use $L = 5$ hidden layers of 25 neurons each. The homogeneous

Dirichlet conditions are imposed *strongly* by multiplying the output by a function that vanishes at the boundary:

$$u_\theta(x) = (x+1)(x-1)\, N_\theta(x). \tag{3.2}$$

Hence $u_\theta(\pm 1) = 0$ exactly, and no boundary penalty term is needed ($C(u_\theta) = 0$ in the notation of [12]). All derivatives u'_θ, u''_θ are computed by automatic differentiation.

Remark 3.1.1. Since tanh is analytic, $u_\theta \in C^\infty([-1,1])$. In particular u''_θ is continuous, while the exact solution of the interface problem has a jump in u'' at $x = 0$ (Remark 2.1.1). A smooth network can therefore approximate u well in H^1, but cannot approximate u'' uniformly near the interface.

3.1.1 An interface-enriched network

To remove the limitation of Remark 3.1.1 we also tested a network with two outputs $N_{\theta,0}$, $N_{\theta,1}$ and

$$u_\theta(x) = (x+1)(x-1)\Big[N_{\theta,0}(x) + \big(\max(x,0)\big)^2 N_{\theta,1}(x)\Big]. \tag{3.3}$$

Proposition 3.1.2. *The function* (3.3) *satisfies* $u_\theta(\pm 1) = 0$*, is continuously differentiable on* $[-1,1]$*, is* C^∞ *on* $[-1,0]$ *and on* $[0,1]$*, and* $[u''_\theta](0) = -2N_{\theta,1}(0)$.

Proof. The function $g(x) = \max(x,0)^2$ is C^1 with $g(0) = g'(0) = 0$, smooth on each side of 0, and g'' jumps from 0 to 2 at $x = 0$. The product rule then gives the regularity claims, and at $x = 0$ the only term of u''_θ that jumps is $(0^2 - 1)\, g''(x)\, N_{\theta,1}(0)$, whose jump is $-2N_{\theta,1}(0)$. □

Thus the enriched network has exactly the regularity of the true solution:

it satisfies both interface conditions (2.4) automatically and can represent the jump (2.5). It uses the a priori knowledge of the interface location, which is available in our problem. This architecture is our own addition and is not part of [12].

3.2 Physics-informed neural networks (PINN)

The PINN loss [11] is the mean squared strong residual at N_c collocation points $x_i \in (-1, 1)$:

$$\mathscr{L}_{\mathrm{PINN}}(u_\theta) = \frac{1}{N_c} \sum_{i=1}^{N_c} \Big(-\varepsilon\, u_\theta''(x_i) + \beta(x_i)\, u_\theta'(x_i) - f(x_i) \Big)^2. \tag{3.4}$$

We use $N_c = 1000$ points drawn uniformly at random and resampled at every iteration. For the interface problem the interface conditions are not part of the loss; with the standard network they hold automatically because u_θ is smooth.

3.3 Variational PINNs (VPINN)

Let $V_M = \mathrm{span}\{\varphi_1, \ldots, \varphi_M\} \subset V$ be a finite dimensional test space. The classical VPINN loss [5], [12, eq. (13)] is

$$\mathscr{L}_{\mathrm{VPINN}}(u_\theta) = \sum_{m=1}^{M} r(u_\theta, \varphi_m)^2, \qquad r(u_\theta, \varphi_m) = (f, \varphi_m) - \varepsilon(u_\theta', \varphi_m') - (\beta u_\theta', \varphi_m). \tag{3.5}$$

Only the first derivative of the network appears. Following [12] we use the sine functions

$$s_m(x) = \sin\Big(m\pi\,\frac{x+1}{2}\Big), \qquad m = 1, \dots, M, \tag{3.6}$$

with $M = 50$, which are the first M eigenfunctions of $-d^2/dx^2$ on $(-1, 1)$ with homogeneous Dirichlet conditions. A drawback of (3.5) is that it depends on the scaling of each φ_m: replacing φ_m by $c_m\varphi_m$ multiplies the m-th term by c_m^2, so the loss is not an intrinsic quantity [12].

3.4 Robust variational PINNs (RVPINN)

RVPINNs [12] fix this problem by measuring the weak residual in the dual norm of V_M. For a given θ, let $\phi \in V_M$ be the Riesz representative of the residual in V_M:

$$(\phi, \varphi_n)_V = r(u_\theta, \varphi_n), \qquad n = 1, \dots, M, \tag{3.7}$$

where $(\cdot,\cdot)_V$ is the inner product inducing (2.8). Writing $\phi = \sum_m \eta_m\varphi_m$, (3.7) becomes the linear system $G\eta = \mathcal{R}(\theta)$ with Gram matrix $G_{nm} = (\varphi_m, \varphi_n)_V$ and residual vector $\mathcal{R}_n(\theta) = r(u_\theta, \varphi_n)$. The RVPINN loss is

$$\mathscr{L}_r^{\phi}(u_\theta) = \|\phi\|_V^2 = r(u_\theta, \phi) = \mathcal{R}(\theta)^T G^{-1}\mathcal{R}(\theta). \tag{3.8}$$

The loss is independent of the basis chosen for V_M: changing basis by an invertible matrix P maps $\mathcal{R} \mapsto P^T\mathcal{R}$ and $G \mapsto P^T G P$, which leaves (3.8) unchanged.

3.4.1 Finite element test functions

For V_M we take the space of continuous piecewise linear functions on a uniform mesh of $M+1$ elements of size $h = 2/(M+1)$, with the standard "hat" basis functions. For the norm (2.8) the Gram matrix is tridiagonal,

$$G = \frac{\varepsilon}{h}\,\mathrm{tridiag}(-1,\, 2,\, -1) \in \mathbb{R}^{M\times M}, \tag{3.9}$$

and we compute G^{-1} explicitly once before training, as in [12]. We use $M = 100$.

3.4.2 Spectral test functions

The sines (3.6) are orthogonal in $(\cdot,\cdot)_V$ with $\|s_m\|_V^2 = \varepsilon m^2\pi^2/4$. Hence the normalised functions $\varphi_m = 2s_m/(\sqrt{\varepsilon}\,\pi m)$ are orthonormal, G is the identity, and (3.8) becomes [12, eq. (54)]

$$\mathscr{L}_r^{\phi}(u_\theta) = \frac{4}{\varepsilon\pi^2}\sum_{m=1}^{M}\frac{1}{m^2}\,r(u_\theta, s_m)^2. \tag{3.10}$$

Comparing with (3.5), the only difference between VPINN and RVPINN with sine test functions is the weight $1/m^2$, which reduces the importance of the high-frequency test functions. This has a direct effect on the speed of training (Chapter 5).

3.5 Error estimates for RVPINNs

The main theoretical results of [12] can be summarised as follows. Let μ and α be the boundedness and inf–sup constants of $a(\cdot,\cdot)$ (Lemmas 2.3.1 and 2.3.2).

Theorem 3.5.1 ([12, Corollary 3])**.** *For every* θ, $\dfrac{1}{\mu}\,\|\phi\|_V \le \|u-u_\theta\|_U$.

Theorem 3.5.2 ([12, Proposition 4])**.** *If there is a local Fortin operator* $\Pi_\theta : V \to V_M$ *with continuity constant* C_Π *(Assumption 1 of [12]), then*

$$\|u-u_\theta\|_U \le \frac{1}{\alpha}\,\mathrm{osc}(u) + \frac{1}{\alpha\, C_\Pi}\,\|\phi\|_V\,, \qquad \mathrm{osc}(u) = \sup_{0\neq v\in V} \frac{a(u, v-\Pi_\theta v)}{\|v\|_V}.$$

Theorem 3.5.1 shows that $\sqrt{\mathscr{L}_r^\phi} = \|\phi\|_V$ is always (up to μ) a lower bound of the energy error, and Theorem 3.5.2 that it is also an upper bound up to an oscillation term that is small when V_M is rich enough. For $\beta = 0$ and $\varepsilon = 1$ one has $\mu = \alpha = 1$, so $\sqrt{\mathscr{L}_r^\phi}$ is essentially *equal* to the error. For our interface problem with $\beta_- > \beta_+$, Lemma 2.3.1 gives $\alpha = 1$, while $\mu = 1+2\beta_-/(\pi\varepsilon) > 1$, so the estimate is still valid but with a wider theoretical band.

3.6 Why the strong residual is not controlled by weak losses

The energy norm (2.8) involves only first derivatives. Because $a(u-u_\theta, v) = r(u_\theta, v)$, the weak losses measure the residual $-\varepsilon u_\theta'' + \beta u_\theta' - f$ in a *negative* norm (the dual norm of $V_M \subset H_0^1$), which is equivalent to the H^1 error. The strong residual, on the other hand, measures the same quantity in L^2, which corresponds to an H^2-type error. A small VPINN/RVPINN loss therefore guarantees a small H^1 error but gives no control of u_θ'' or of the strong residual. Moreover:

- high-frequency components of the residual are only weakly penalised, and in RVPINN with sines they are further damped by the weight $1/m^2$

in (3.10);

- every test function vanishes at $x = \pm 1$, so the losses are least sensitive to the residual close to the boundary;
- in the interface problem u'' is discontinuous, which a smooth network cannot reproduce (Remark 3.1.1).

These observations explain the behaviour of u''_θ reported in Chapters 4 and 5.

3.7 Implementation details

The methods were implemented in a single Python file using PyTorch [10] in double precision. The main choices are:

- **Quadrature.** All integrals in (3.5)–(3.8) are computed with composite 5-point Gauss–Legendre quadrature. The breakpoints always include the interface $x = 0$, so that the discontinuous coefficient β is integrated exactly on each side. For the sine test functions we use 400 sub-intervals on each half of the domain (4000 points in total). For the hat functions we use one sub-interval per mesh element; the element containing $x = 0$ (when $M + 1$ is odd) is split at $x = 0$.
- **Optimiser.** Adam [7] with learning rate 5×10^{-4}, as in [12], for 6000 iterations (the protocol of [12]) and, in a second experiment, 20000 iterations.
- **Model selection.** The reported network is the one with the *lowest training loss* over all iterations. The exact solution is never used for training or for selecting the model; it is only used to compute errors.

- **Error measures.** Using a fine quadrature (2000 elements, 10^4 points) we compute the relative energy error $e_U = \|u - u_\theta\|_U \,/\, \|u\|_U$, the relative L^2 error $e_{L^2} = \|u - u_\theta\|_{L^2} \,/\, \|u\|_{L^2}$, the relative second-derivative error $e_2 = \|u'' - u_\theta''\|_{L^2} \,/\, \|u''\|_{L^2}$ (computed piecewise on $(-1, 0)$ and $(0, 1)$), and the root-mean-square strong residual, also restricted to the regions near the boundary ($|x| > 0.9$), near the interface ($|x| < 0.1$) and in the bulk.

- **Initialisation.** PyTorch default initialisation with a fixed random seed, so that all methods start from the same network.

The training procedure common to all methods is summarised in Algorithm 1.

Algorithm 1 Training procedure used for PINN, VPINN and RVPINN

Pre-compute quadrature points and weights, the test functions φ_m, φ_m' at those points, the load vector (f, φ_m) and G^{-1}
Initialise θ; $\mathscr{L}_{\min} \leftarrow \infty$
for $k = 1, \dots, K$ **do**
    Evaluate u_θ, u_θ' (and u_θ'' for PINN) by automatic differentiation
    Compute the loss (3.4), (3.5), (3.8) or (3.10)
    Compute $\nabla_\theta \mathscr{L}$ by back-propagation and take one Adam step
    if $\mathscr{L} < \mathscr{L}_{\min}$ **then** store θ and set $\mathscr{L}_{\min} \leftarrow \mathscr{L}$
    end if
end for
return the stored θ with the smallest loss

Chapter 4

Results I: Pure Diffusion Benchmark ($\beta = 0$)

Before treating the discontinuous coefficient, we validated the implementation on the smooth benchmark of [12, Section 5.4]:

$$-u'' = f \ \text{ in } (-1,1), \quad u(\pm 1) = 0, \qquad u(x) = x \sin\big(\pi(x+1)\big), \tag{4.1}$$

i.e. $\varepsilon = 1$, $\beta = 0$ and $f = -u'' = 2\pi\cos(\pi x) - \pi^2 x \sin(\pi x)$. For this problem $\mu = \alpha = 1$, so by Section 3.5 the RVPINN loss should satisfy $\sqrt{\mathscr{L}_r^\phi} \approx \|u - u_\theta\|_U$. All four methods were trained for 6000 iterations with the settings of Section 3.7.

4.1 Accuracy

Table 4.1 lists the error measures of the selected (lowest-loss) networks, and Figure 4.1 compares the solutions, the pointwise errors, the second-derivative errors and the convergence histories.

Table 4.1: Pure diffusion benchmark, 6000 iterations: relative errors in the energy norm, L^2 norm and second derivative, and RMS of the strong residual (overall and in the boundary region $|x| > 0.9$ and the bulk $0.1 \leq |x| \leq 0.9$).

Method	e_U	e_{L^2}	e_2	res. (all)	res. ($\|x\| > 0.9$)	res. (bulk)
PINN	2.7×10^{-4}	3.9×10^{-4}	8.3×10^{-4}	5.4×10^{-3}	1.1×10^{-2}	4.0×10^{-3}
VPINN (sines)	1.9×10^{-4}	1.1×10^{-4}	1.1×10^{-3}	7.4×10^{-3}	2.0×10^{-2}	3.7×10^{-3}
RVPINN (FE hats)	2.7×10^{-3}	4.0×10^{-4}	2.2×10^{-2}	1.4×10^{-1}	4.2×10^{-1}	5.9×10^{-2}
RVPINN (spectral)	2.6×10^{-3}	4.6×10^{-4}	2.2×10^{-2}	1.4×10^{-1}	4.2×10^{-1}	5.9×10^{-2}

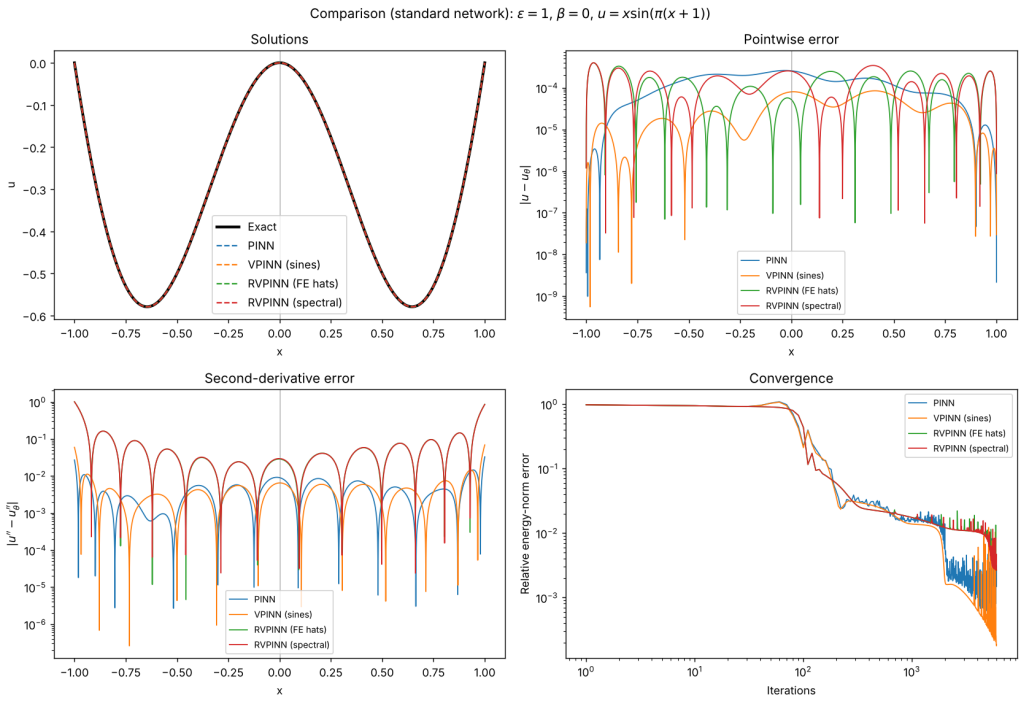


Figure 4.1: Pure diffusion benchmark: solutions, pointwise error, second-derivative error and relative energy-norm error during training for the four methods.

All methods approximate the solution very well: the curves in the top-left panel of Figure 4.1 cannot be distinguished from the exact solution, and

all relative L^2 errors are below 5×10^{-4}. After 6000 iterations PINN and VPINN reach a smaller energy error ($\approx 2 \times 10^{-4}$) than the two RVPINNs ($\approx 2.6 \times 10^{-3}$). The convergence history shows why: PINN and VPINN have a sharp drop of the error at around 2000 iterations, whereas the RVPINN error decreases more slowly and is still decreasing at the end of training. The final RVPINN error is of the same order as the one reported in [12, Figure 1]. The two RVPINN variants (FE hats and sines) behave almost identically, which illustrates that the RVPINN loss does not depend on the choice of the test space as long as it is rich enough.

As a cross-check, an independent implementation of RVPINN with spectral test functions written independently by the project partner (using the trapezoidal rule with 4000 nodes, as in [12], and a network with four hidden layers) gave a relative energy error of 2.0×10^{-3} and a relative L^2 error of 3.7×10^{-4} after 6000 iterations, in agreement with Table 4.1.

4.2 The RVPINN loss as an error estimator

Figure 4.2 shows the RVPINN (spectral) result in detail, and Figure 4.3 compares the true energy error with the loss-based estimate $\sqrt{\mathscr{L}} / \|u\|_U$ along the whole training history.

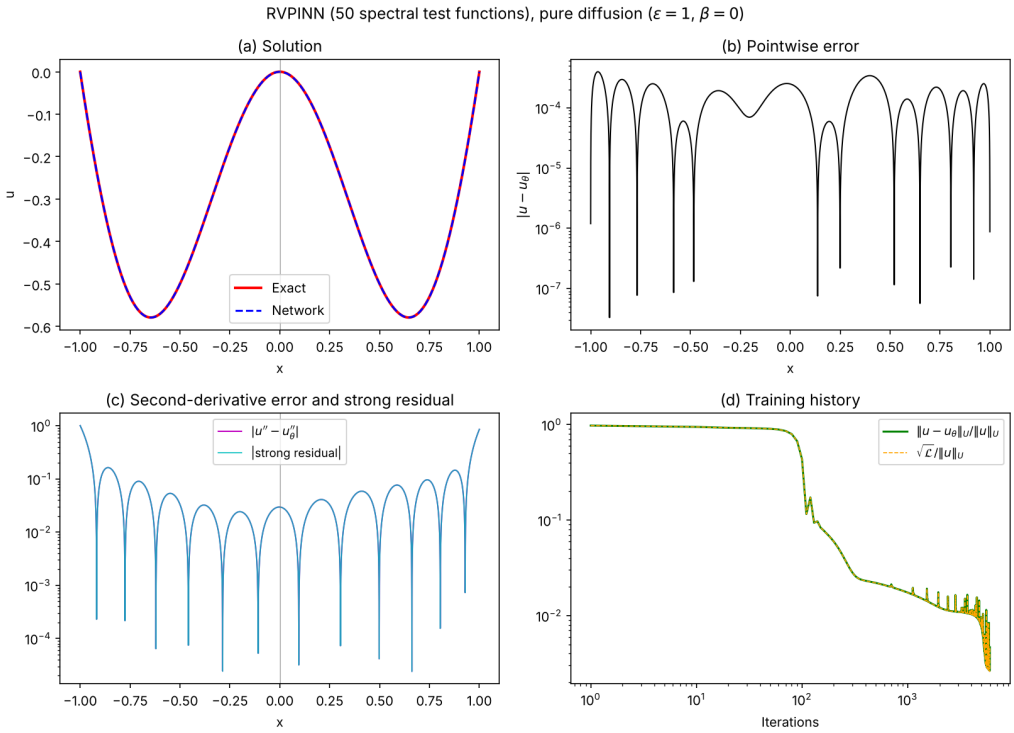


Figure 4.2: RVPINN with 50 spectral test functions for the pure diffusion benchmark: (a) solution, (b) pointwise error, (c) second-derivative error and strong residual (identical here since $\varepsilon = 1$, $\beta = 0$), (d) true relative energy error and loss-based estimate during training.

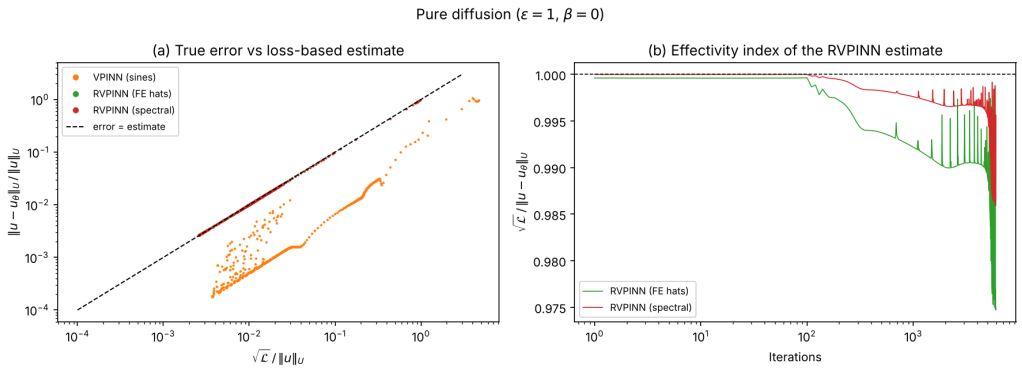


Figure 4.3: Pure diffusion benchmark. (a) True relative energy error against $\sqrt{\mathscr{L}} / \|u\|_U$ at every 10th iteration; the dashed line is "error = estimate". (b) Effectivity index $\sqrt{\mathscr{L}} / \|u - u_\theta\|_U$ of the RVPINN losses.

For both RVPINNs the points lie exactly on the diagonal: the effectivity index $\sqrt{\mathscr{L}_r^\phi} / \|u - u_\theta\|_U$ stays between 0.975 and 1 during the entire training, and equals 0.986 (spectral) and 0.975 (FE) for the selected networks. This reproduces Figure 1 of [12] and confirms Theorems 3.5.1 and 3.5.2: the loss is a reliable and efficient estimate of the error, and since it never exceeds the error, it is a guaranteed lower bound. For the classical VPINN loss, in contrast, $\sqrt{\mathscr{L}_{\text{VPINN}}}$ is 10–20 times larger than the error and the ratio changes during training, so it cannot be used as an error indicator even though the VPINN solution itself is accurate.

4.3 Second derivative and strong residual

The last columns of Table 4.1 and panel (c) of Figure 4.2 show the behaviour that motivated part of this project: although the RVPINN solution is accurate in H^1, the error in u_θ'' (equal to the strong residual for this problem) is about 2% in relative terms, and it is largest near the boundaries, where its

RMS value is about seven times larger than in the bulk (0.42 against 0.059). The second-derivative error has an oscillatory structure with roughly twenty oscillations over the domain.

This is consistent with Section 3.6. The RVPINN loss controls the residual only in the dual norm, in which a residual component of frequency m is weighted by $1/m^2$ (3.10). High-frequency errors of the second derivative are therefore cheap in terms of the loss and are removed only slowly by the optimiser. Near $x = \pm 1$ all test functions vanish, so the loss is least sensitive there. PINN, which minimises the strong residual directly, has a second-derivative error more than twenty times smaller, and VPINN, which weights all frequencies equally, lies in between in the boundary region. This does not indicate an error in the implementation: it is the expected consequence of measuring the residual in a weak norm.

Chapter 5

Results II: Discontinuous Advection Coefficient

In this chapter we apply the methods to the interface problem (2.3)–(2.4) with a piecewise constant advection coefficient, which is the main task of the project.

5.1 Test problem and exact solution

We take

$$\varepsilon = 0.1, \qquad \beta_- = 2, \qquad \beta_+ = 1, \qquad f \equiv 1. \tag{5.1}$$

Since $\beta_- > \beta_+$, Lemma 2.3.1 gives $\alpha = 1$, and Lemma 2.3.2 gives $\mu = 1+2\beta_-/(\pi\varepsilon) \approx 13.7$. The Péclet numbers $\beta_\pm/\varepsilon$ are 20 and 10, so the problem is moderately advection dominated.

Exact solution. On each subinterval, (2.3) with constant β and f is a linear ODE with constant coefficients, whose general solution is $A + Be^{\beta x/\varepsilon} + fx/\beta$. To avoid overflow we write the exponential relative to the outflow end

of each subinterval:

$$u(x) = \begin{cases} A_- + B_-\, e^{\beta_- x/\varepsilon} + \dfrac{x}{\beta_-}, & -1 \le x \le 0, \\[2ex] A_+ + B_+\, e^{\beta_+ (x-1)/\varepsilon} + \dfrac{x}{\beta_+}, & 0 \le x \le 1. \end{cases} \tag{5.2}$$

The four constants are determined by the four conditions $u(-1) = 0$, $u(1) = 0$, $u(0^-) = u(0^+)$ and $u'(0^-) = u'(0^+)$, which form a 4×4 linear system. Solving it numerically gives

$$A_- \approx 0.500000, \quad B_- \approx 0.0249654, \quad A_+ \approx 0.525035, \quad B_+ \approx -1.525035.$$

The code verifies at start-up that (5.2) satisfies the differential equation, the boundary conditions and both interface conditions up to rounding errors (residuals of order 10^{-11} or smaller).

The solution and its first two derivatives are shown in Figure 5.1. It has two features that make it a challenging test:

- a **boundary layer** at the outflow boundary $x = 1$, of width $\varepsilon/\beta_+ = 0.1$, where u drops from its maximum $u \approx 1.153$ (at $x \approx 0.728$) to zero;
- an **interface layer**: on $(-1, 0)$ the flow goes towards the interface, so the left solution has an exponential layer of width $\varepsilon/\beta_- = 0.05$ just to the left of $x = 0$. Here u'' grows from almost zero to $u''(0^-) \approx 9.986$, and then jumps to $u''(0^+) \approx -0.007$. The jump, $[u''](0) \approx -9.993$, agrees with formula (2.5): $(\beta_+ - \beta_-)u'(0)/\varepsilon = -10 \times 0.99931$.

Since u' is continuous and piecewise smooth, u belongs to $H^2(-1, 1)$, but $u \notin C^2([-1, 1])$: its second derivative is discontinuous at $x = 0$ and therefore cannot be approximated uniformly by the second derivatives of smooth functions such as a tanh network.

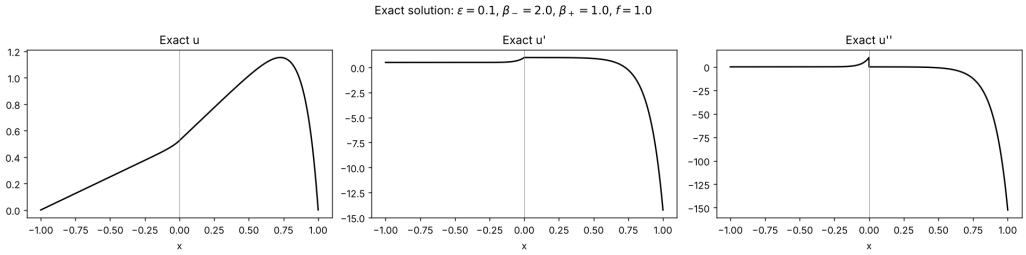


Figure 5.1: Exact solution of the interface problem with $\varepsilon = 0.1$, $\beta_- = 2$, $\beta_+ = 1$, $f = 1$: u, u' and u''. The vertical line marks the interface $x = 0$.

5.2 Results with the standard network

All four methods were trained for 20000 iterations; the best (lowest-loss) model after the first 6000 iterations was also recorded, so that the protocol of [12] and the longer training can be compared. The results are listed in Table 5.1 and shown in Figures 5.2 and 5.3.

Table 5.1: Discontinuous advection, standard network: relative errors and RMS strong residual near the boundary ($|x| > 0.9$), near the interface ($|x| < 0.1$) and in the bulk, after 6000 and 20000 iterations.

Method	Its.	e_U	e_{L^2}	e_2	res. $\lvert x\rvert > 0.9$	res. $\lvert x\rvert < 0.1$	res. bulk
PINN	6000	3.1×10^{-2}	2.8×10^{-2}	4.3×10^{-2}	3.6×10^{-2}	2.9×10^{-1}	5.8×10^{-2}
	20000	1.5×10^{-2}	8.5×10^{-3}	3.6×10^{-2}	2.7×10^{-2}	2.6×10^{-1}	3.1×10^{-2}
VPINN (sines)	6000	7.9×10^{-3}	4.8×10^{-3}	6.6×10^{-2}	4.3×10^{-1}	2.0×10^{-1}	1.9×10^{-2}
	20000	6.3×10^{-3}	1.6×10^{-4}	1.4×10^{-1}	1.0×10^{0}	1.2×10^{-1}	4.1×10^{-3}
RVPINN (FE hats)	6000	2.2×10^{-2}	7.4×10^{-3}	1.5×10^{-1}	1.1×10^{0}	2.9×10^{-1}	5.3×10^{-2}
	20000	1.0×10^{-2}	3.4×10^{-4}	1.9×10^{-1}	1.4×10^{0}	1.7×10^{-1}	1.4×10^{-2}
RVPINN (spectral)	6000	2.3×10^{-2}	7.2×10^{-3}	1.7×10^{-1}	1.2×10^{0}	2.9×10^{-1}	5.3×10^{-2}
	20000	1.7×10^{-2}	4.7×10^{-4}	2.7×10^{-1}	2.0×10^{0}	1.7×10^{-1}	1.3×10^{-2}

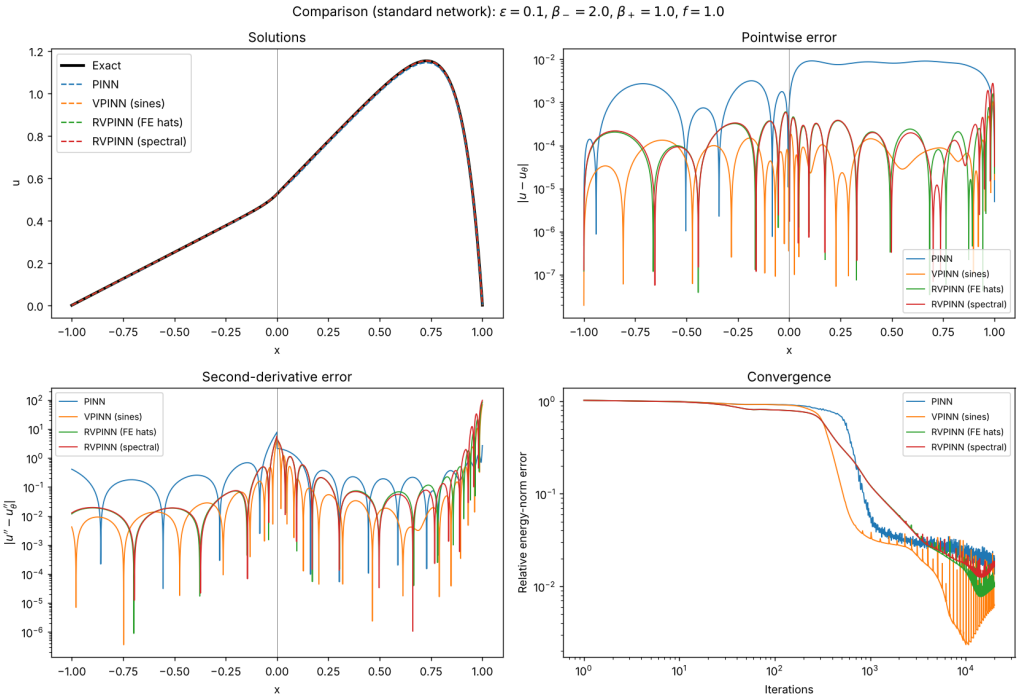


Figure 5.2: Discontinuous advection, standard network, 20000 iterations: solutions, pointwise error, second-derivative error and convergence of the relative energy error.

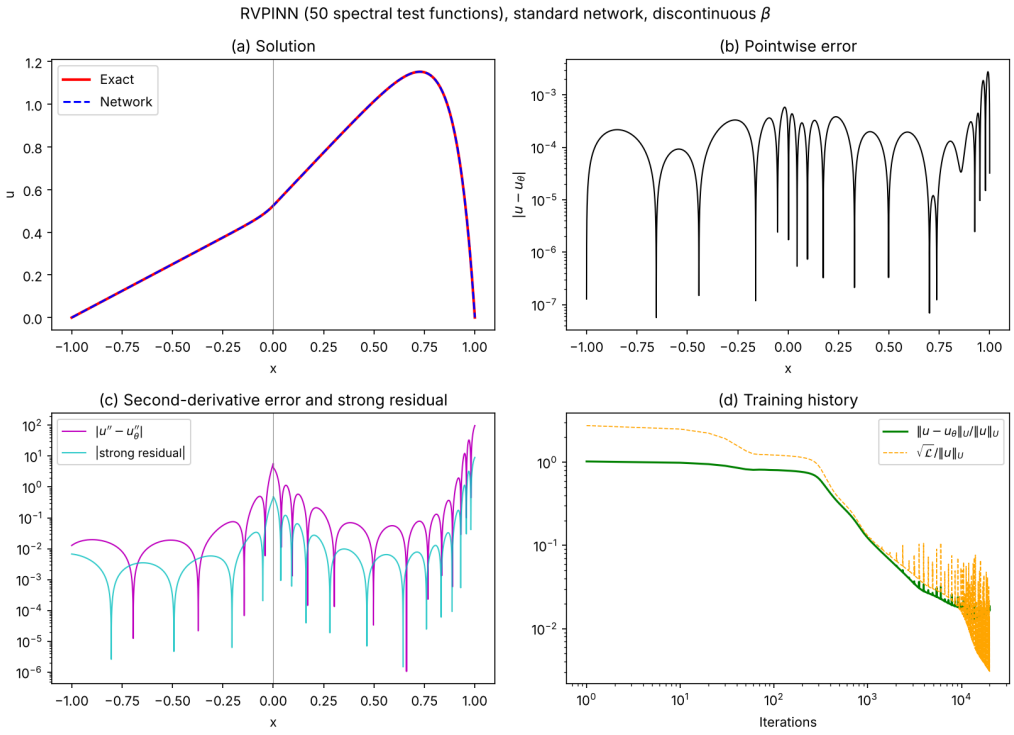


Figure 5.3: RVPINN with 50 spectral test functions, standard network, 20000 iterations: (a) solution, (b) pointwise error, (c) second-derivative error and strong residual, (d) true error and loss-based estimate.

Accuracy. All methods capture the solution, including the boundary layer and the kink at the interface (Figure 5.2, top left). With the training protocol of [12] (6000 iterations) VPINN is the most accurate method in the energy norm ($e_U \approx 7.9 \times 10^{-3}$), followed by the two RVPINNs ($\approx 2.2 \times 10^{-2}$) and PINN ($3.1 \times 10^{-2}$). As in Chapter 4, the RVPINN error is still decreasing at 6000 iterations, because the weight $1/m^2$ makes the fine-scale features (here the two layers) slow to learn. With 20000 iterations the RVPINN L^2 errors drop by more than an order of magnitude, to 3–5 $\times 10^{-4}$, but the energy errors only reach 1.0–1.7 $\times 10^{-2}$. PINN has by far the largest L^2

error (8.5×10^{-3} after 20000 iterations): its strong residual is dominated by the interface region (RMS 0.26 for $|x| < 0.1$, about ten times the value elsewhere), where the smooth network cannot satisfy the equation on both sides at once.

Second derivative and strong residual. For all three variational methods the second-derivative error and the strong residual are concentrated in the boundary layer at $x = 1$ and, to a lesser extent, at the interface (Figure 5.2, bottom left, and Figure 5.3(c)). For example, for RVPINN (spectral) after 20000 iterations the RMS strong residual is 2.0 for $|x| > 0.9$, 0.17 for $|x| < 0.1$ and only 0.013 in the bulk. This is exactly the behaviour observed in our first experiments and is explained by Section 3.6: in the boundary layer $|u''|$ reaches about 150 (Figure 5.1), all test functions vanish at $x = 1$, and the weak losses do not penalise high-frequency residuals. Note also that, for VPINN and RVPINN, the second-derivative error *increases* between 6000 and 20000 iterations while the L^2 error decreases: training continues to reduce the weak residual, and the weak residual simply does not see these errors.

5.3 Reliability of the RVPINN error estimate

Figure 5.4 compares the loss-based estimate with the true error during training. For the first $\approx 10^4$ iterations the RVPINN estimate behaves as predicted by Theorems 3.5.1 and 3.5.2: the effectivity index decreases from about 2.7 to between 1 and 1.3 and the points follow the diagonal. Values larger than one are allowed by Theorem 3.5.1, since $\mu \approx 13.7 > 1$ for this problem (in contrast to the pure diffusion case where $\mu = 1$). At 6000 iterations the effectivity index is 1.27 (FE) and 1.20 (spectral).

After about 10^4 iterations, however, the estimate separates from the error: the loss keeps decreasing, but the true error stays at $\approx 10^{-2}$, and the effectivity index of the selected networks falls to 0.33 (FE) and 0.18 (spectral). The estimate is then no longer an upper bound. This is the situation described by the oscillation term in Theorem 3.5.2 and illustrated in [12, Figure 3]: the network has (approximately) solved the discrete Petrov–Galerkin problem $r(u_\theta, v_M) = 0$ for all $v_M \in V_M$, and the remaining error lies in the part that V_M cannot detect. With 50 sines or 100 hats, the test functions are too coarse to resolve residuals in a boundary layer of width 0.1 and an interface layer of width 0.05. The same effect is visible for VPINN, whose energy error reaches a minimum of about 2.5×10^{-3} near 10^4 iterations and then increases again (Figure 5.2, bottom right). A consequence is that selecting the network with the smallest loss does not give the network with the smallest error once this regime is reached.

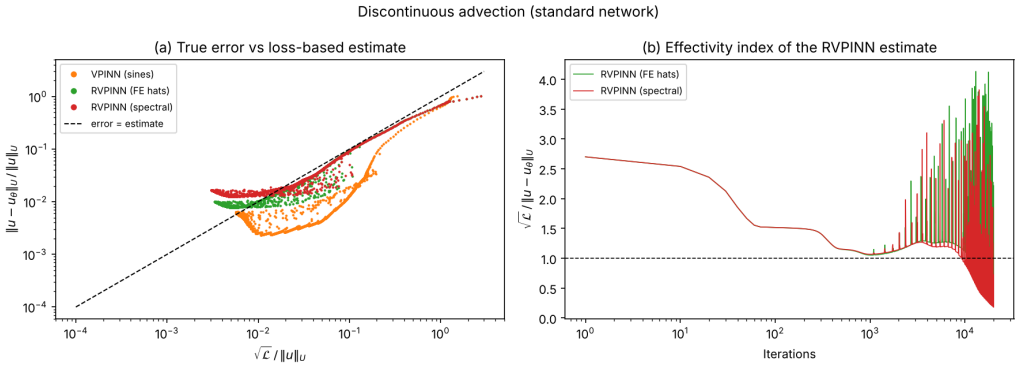


Figure 5.4: Discontinuous advection, standard network, $M = 50$ sines and $M = 100$ hats: (a) true relative energy error against the loss-based estimate, (b) effectivity index of the RVPINN losses.

Richer test spaces. To test this explanation we repeated the two RVPINN runs with $M = 200$ sine functions and $M = 400$ hat functions, keeping

everything else unchanged. Table 5.2 and Figure 5.5 show the result: the effectivity index now stays above one for the whole training and equals 1.01–1.03 for the selected networks, and the energy error improves to 3.5×10^{-3}, three to five times smaller than with the coarse test spaces and now better than VPINN with 50 sines. This confirms that the loss of robustness was caused by an insufficient test space and not by the discontinuity of β itself.

Table 5.2: RVPINN with the standard network after 20000 iterations: effect of the size M of the test space on the energy error and on the effectivity index $\sqrt{\mathscr{L}} / \|u - u_\theta\|_U$.

Test space	M	e_U	e_{L^2}	effectivity index
FE hats	100	1.0×10^{-2}	3.4×10^{-4}	0.33
FE hats	400	3.6×10^{-3}	3.1×10^{-4}	1.03
sines	50	1.7×10^{-2}	4.7×10^{-4}	0.18
sines	200	3.5×10^{-3}	2.9×10^{-4}	1.01

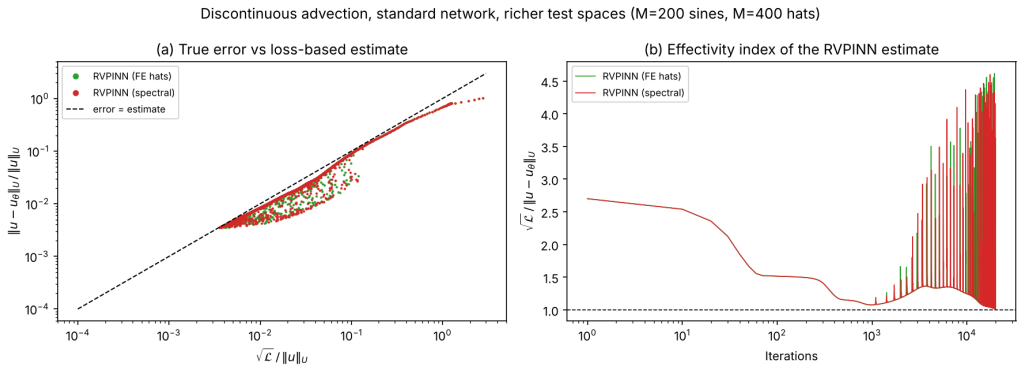


Figure 5.5: As Figure 5.4, but with $M = 200$ sines and $M = 400$ hats. The estimate remains reliable during the whole training.

The occasional sharp spikes in the effectivity index in Figures 5.4 and 5.5

are caused by short loss spikes of the Adam optimiser with a fixed learning rate, which also produce the oscillations visible in the convergence curves; a decaying learning rate would remove them.

5.4 Results with the interface-enriched network

Finally, we repeated all experiments with the interface-enriched network (3.3), which can represent the jump of u'' at $x = 0$. The results are given in Table 5.3 and Figure 5.6.

Table 5.3: Discontinuous advection, interface-enriched network (test spaces $M = 50$ sines, $M = 100$ hats). The last column is the effectivity index of the loss-based estimate (meaningful for RVPINN only).

Method	Its.	e_U	e_{L^2}	e_2	res. $\lvert x\rvert < 0.1$	eff. index
PINN	6000	6.8×10^{-4}	3.1×10^{-4}	3.5×10^{-3}	2.5×10^{-2}	–
	20000	5.0×10^{-4}	2.5×10^{-4}	1.5×10^{-3}	1.1×10^{-2}	–
VPINN (sines)	6000	7.6×10^{-4}	8.0×10^{-5}	1.5×10^{-2}	4.8×10^{-2}	–
	20000	5.4×10^{-4}	1.9×10^{-5}	1.2×10^{-2}	4.1×10^{-2}	–
RVPINN (FE hats)	6000	1.6×10^{-2}	4.8×10^{-3}	7.6×10^{-2}	2.5×10^{-1}	1.24
	20000	4.6×10^{-3}	5.3×10^{-4}	3.3×10^{-2}	1.5×10^{-1}	1.07
RVPINN (spectral)	6000	1.6×10^{-2}	5.4×10^{-3}	7.8×10^{-2}	2.5×10^{-1}	1.24
	20000	4.7×10^{-3}	5.2×10^{-4}	3.7×10^{-2}	1.5×10^{-1}	1.03

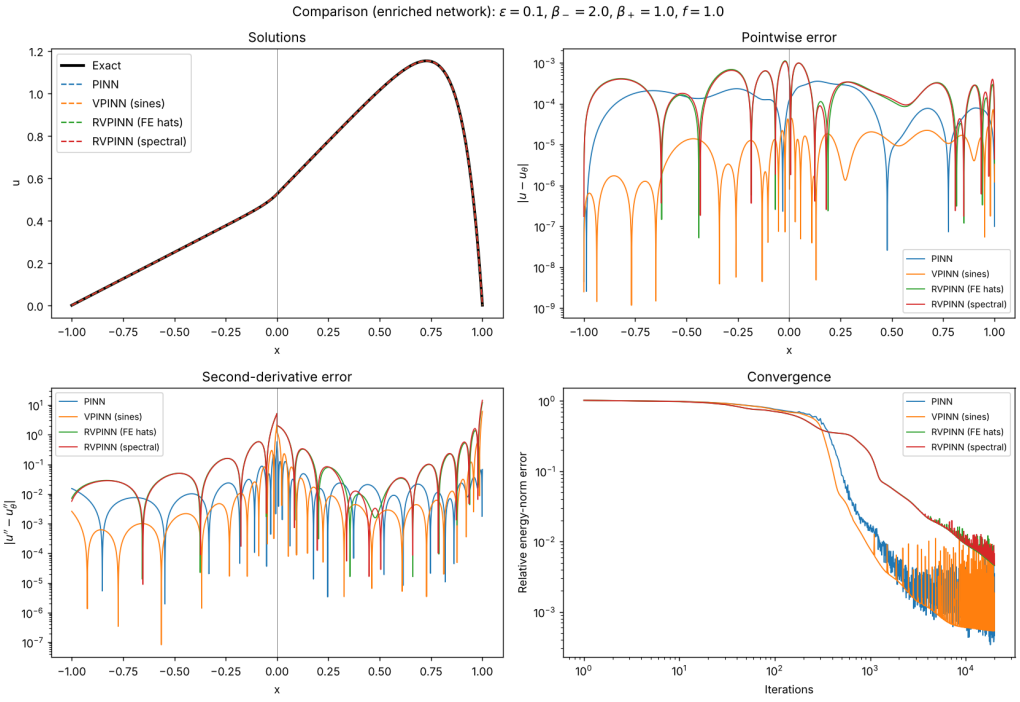


Figure 5.6: Discontinuous advection, interface-enriched network, 20000 iterations: solutions, pointwise error, second-derivative error and convergence.

The enrichment has a large effect on the methods that reached the limit of the standard network. Compared with Table 5.1 (20000 iterations):

- the PINN energy error decreases from 1.5×10^{-2} to 5.0×10^{-4} (a factor of 30), its second-derivative error from 3.6% to 0.15%, and its strong residual near the interface from 0.26 to 0.011: once the network can represent the jump of u'', PINN is no longer limited by the interface;
- the VPINN energy error decreases from 6.3×10^{-3} to 5.4×10^{-4} (a factor of 12), and its L^2 error to 1.9×10^{-5}, the smallest of all experiments;
- the RVPINN errors improve from 1.0–1.7×10^{-2} to 4.6×10^{-3}, and the estimate stays reliable (effectivity index 1.03–1.07), but RVPINN

is still converging at 20000 iterations and remains less accurate than PINN and VPINN for the same number of iterations.

The second-derivative errors of the variational methods are still largest at the boundary layer, which the enrichment does not address (Figure 5.6, bottom left).

5.5 Discussion

The experiments lead to the following observations.

1. All methods can solve the interface problem. Because the interface conditions are natural in the weak form (Proposition 2.2.2), the variational methods require no special treatment of the interface, apart from placing a quadrature breakpoint at $x = 0$.

2. The RVPINN loss remains a reliable and efficient estimate of the energy error for a discontinuous β, provided the test space is rich enough to resolve the layers of the solution. With a coarse test space the estimate becomes too optimistic late in training, as predicted by the oscillation term of Theorem 3.5.2. The classical VPINN and PINN losses never provide such an estimate: in our experiments the ratio $\sqrt{\mathscr{L}} / \|u - u_\theta\|_U$ for these losses ranged from about 0.9 to 20, depending on the problem, the network and the stage of training.

3. For a fixed number of iterations RVPINN converges more slowly than VPINN, because its loss damps high-frequency residuals by $1/m^2$. Its advantage is the error control, not the speed.

4. Large errors in u''_θ and in the strong residual near the boundary layer and the interface are expected for variational methods, since their losses

control only the H^1 error. They are not a sign of an implementation error. If the second derivative is important, the network should be able to represent the correct regularity (as the enriched network does at the interface), or the strong residual should be added to the loss.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

In this project we studied physics-informed neural networks (PINNs), variational PINNs (VPINNs) and robust variational PINNs (RVPINNs), and implemented all three for the one-dimensional diffusion–advection problem of [12], first with $\beta = 0$ and then with a piecewise constant, discontinuous advection coefficient. The main conclusions are the following.

1. **Weak formulation of the interface problem.** The interface conditions (continuity of the solution and of the diffusive flux) are natural conditions of the weak formulation. When β jumps at $x = 0$, the advection term contributes $(\beta_- - \beta_+)v(0)^2/2$ to $a(v, v)$, so the coercivity used in [12] holds with constant one if $\beta_- \geq \beta_+$, and with a smaller constant if $0 < \beta_+ - \beta_- < 4\varepsilon$ (Lemma 2.3.1).

2. **Validation.** For the pure diffusion benchmark all methods reach relative errors below 3×10^{-3} in the energy norm, and the RVPINN loss reproduces the true error with an effectivity index between 0.975 and

1 throughout training, as reported in [12].

3. **Discontinuous advection.** All methods approximate the solution of the interface problem, including its boundary layer and its interface layer. With the standard network, VPINN gives the smallest energy error for a fixed number of iterations; RVPINN converges more slowly because its loss damps high-frequency residuals.

4. **Robustness of the RVPINN estimate.** The RVPINN loss is a reliable estimate of the energy error also for a discontinuous β, as long as the test space resolves the layers of the solution. With 50 sines or 100 hats the estimate becomes too optimistic after about 10^4 iterations; with 200 sines or 400 hats it remains reliable (effectivity index 1.01–1.03) and the error improves to 3.5×10^{-3}.

5. **Second derivative and strong residual.** The large errors in u_θ'' and in the strong residual near the boundary layer and the interface are a property of variational losses, which control only the H^1 error, and of smooth networks, which cannot represent a jump in u''. They do not indicate an implementation error.

6. **Interface-enriched network.** Building the known regularity of the solution into the network, through the term $\max(x, 0)^2 N_{\theta,1}(x)$, reduces the energy error of PINN to 5.0×10^{-4} and of VPINN to 5.4×10^{-4}, and reduces the strong residual of PINN at the interface by a factor of more than twenty.

6.2 Future work

Several directions follow naturally from this work:

- **Smaller diffusion.** Repeating the study for smaller ε (stronger advection dominance), including the case $\beta_+ - \beta_- \geq 4\varepsilon$, in which the bilinear form is not coercive and the inf–sup constant has to be estimated separately.

- **Adaptive or layer-adapted test spaces.** Since the robustness of RVPINN depends on the test space resolving the layers, graded meshes or adaptively refined finite element test functions could give reliable estimates at a lower cost than uniform refinement.

- **Optimisation.** A decaying learning rate, or a second-order optimiser such as L-BFGS in the final phase of training, would remove the loss spikes of Adam and may accelerate the slow convergence of RVPINN.

- **Weak imposition of boundary conditions** as in [12, Section 5.2], and extension to two-dimensional interface problems with discontinuous coefficients.

- **Combined losses.** Adding a small multiple of the strong residual (away from the interface) to the RVPINN loss could control the second derivative in the boundary layer while keeping the error estimate.

Appendix A

Code and Reproducibility

The single Python file `btp_interface.py` produced all results in this report. It requires `torch`, `numpy` and `matplotlib`. The file contains the exact solutions, the network architectures (standard and interface-enriched), the four loss functions (PINN, VPINN, RVPINN with finite element and spectral test functions), the quadrature rules, the training loop and all plotting routines.

Usage

```
# discontinuous advection, all methods, both networks
python btp_interface.py --epochs 20000

# richer test spaces for RVPINN
python btp_interface.py --epochs 20000 --nets standard \
       --methods rvpinn_fem,rvpinn_sp --M_spec 200 --M_fem 400

# pure diffusion benchmark (beta = 0)
python btp_interface.py --case smooth --nets standard \
```

```
    --epochs 6000

# change the data of the interface problem
python btp_interface.py --eps 0.05 --beta_minus 3 --beta_plus 1

# only one method / one network
python btp_interface.py --methods rvpinn_sp --nets standard
```

Other options are `--M_spec` (number of sine test functions, default 50), `--M_fem` (number of hat test functions, default 100), `--n_col` (PINN collocation points, default 1000), `--lr` (default 5×10^{-4}), `--seed` and `--report_at` (an intermediate iteration at which the best model so far is also evaluated, default 6000).

For every run the script writes a four-panel figure (solution, pointwise error, second-derivative error and strong residual, training history), a comparison figure for all methods, the training history as a `.npz` file and a table `results.csv` with all error measures. At start-up it also checks that the exact solution satisfies the differential equation, the boundary conditions and the interface conditions to machine precision.

All computations were run on a CPU in double precision; one run of 20000 iterations takes between about one and seven minutes.

Bibliography

[1] S. Berrone, C. Canuto, and M. Pintore. Variational physics informed neural networks: the role of quadratures and test functions. *Journal of Scientific Computing*, 92:100, 2022.

[2] S. C. Brenner and L. R. Scott. *The Mathematical Theory of Finite Element Methods*. Springer, New York, third edition, 2008.

[3] S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis. Physics-informed neural networks (PINNs) for fluid mechanics: A review. *Acta Mechanica Sinica*, 37:1727–1738, 2021.

[4] L. C. Evans. *Partial Differential Equations*, volume 19 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, second edition, 2010.

[5] E. Kharazmi, Z. Zhang, and G. E. Karniadakis. Variational physics-informed neural networks for solving partial differential equations. *arXiv preprint arXiv:1912.00873*, 2019.

[6] E. Kharazmi, Z. Zhang, and G. E. Karniadakis. hp-VPINNs: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, 374:113547, 2021.

[7] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[8] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, and M. W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems*, 34:26548–26560, 2021.

[9] I. E. Lagaris, A. Likas, and D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[10] A. Paszke, S. Gross, F. Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[11] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[12] S. Rojas, P. Maczuga, J. Muñoz-Matute, D. Pardo, and M. Paszyński. Robust variational physics-informed neural networks. *arXiv preprint arXiv:2308.16910v3*, 2024.